

Documentação do Transpilador Latim-Python

Transpilador Latim-Python

Projeto desenvolvido para a disciplina de **Teoria da Computação e Compiladores**. O sistema implementa um transpilador capaz de receber programas escritos em uma linguagem didática inspirada em termos em latim e gerar código equivalente em **Python 3**.

O projeto contempla as principais etapas estudadas em compiladores: definição de gramática, análise sintática, construção de uma árvore de sintaxe abstrata, validação semântica e geração de código.

1. Objetivo do Projeto

O objetivo é criar uma linguagem de programação simples, chamada neste projeto de **Latim**, e desenvolver um transpilador que converta seus comandos para Python.

A linguagem permite:

- declaração de variáveis;
- atribuição de valores;
- uso de tipos primitivos e listas;
- operações matemáticas com precedência;
- comandos de entrada e saída;
- estruturas condicionais;
- estruturas de repetição;
- validação semântica antes da geração do código final.

2. Tecnologias Utilizadas

Tecnologia	Função no projeto
Python 3	Linguagem principal do transpilador
Lark	Biblioteca usada para definir e processar a gramática
Tkinter	Biblioteca usada para criar a interface gráfica
Pytest	Ferramenta usada para testes automatizados

As dependências externas estão listadas em requirements.txt:

```
lark
pytest
```

3. Estrutura de Pastas e Arquivos

```
transpilador-latim-main/
├── main.py
├── gui.py
```

```

├── requirements.txt
├── README.md
├── programa.latim
├── saida.py
├── erro_semantico.latim
├── teste_decimal.latim
├── teste_erro.latim
├── teste_if.latim
├── teste_io.latim
├── teste_loops.latim
├── teste_math.latim
├── teste_soma.latim
├── src/
│   ├── grammar.lark
│   ├── models.py
│   ├── parser_service.py
│   ├── semantic_service.py
│   └── generator_service.py
├── tests/
│   └── test_parser.py

```

Arquivos principais

Arquivo	Responsabilidade
main.py	Entrada principal via terminal. Lê um arquivo .latim, valida e gera saida.py.
gui.py	Interface gráfica com editor, console e visualização do Python gerado.
src/grammar.lark	Define a gramática da linguagem Latim.
src/models.py	Define as classes da AST usando dataclass.
src/parser_service.py	Converte a árvore gerada pelo Lark em uma AST própria.
src/semantic_service.py	Valida regras semânticas, como declaração de variáveis e compatibilidade de tipos.
src/generator_service.py	Gera o código Python equivalente.
tests/test_parser.py	Testes automatizados da análise sintática e da construção da AST.
.	

4. Funcionamento Geral

O fluxo do transpilador é dividido em quatro etapas principais:

Código .latim

↓

Análise léxica e sintática

↓

Construção da AST

↓

Análise semântica



Geração de código Python



Execução do arquivo gerado

4.1 Análise léxica e sintática

A gramática da linguagem está no arquivo `src/grammar.lark`. Ela define as regras de formação dos programas em Latim, incluindo comandos, expressões, tipos, operadores e estruturas de controle.

O programa deve começar com:

`principium`

E terminar com:

`finis`

4.2 Construção da AST

O arquivo `src/parser_service.py` usa um Transformer do Lark para converter o resultado da análise sintática em objetos Python definidos em `src/models.py`.

Exemplos de nós da AST:

- Programa
- Bloco
- Declaracao
- Atribuicao
- Leitura
- Escrita
- IfStmt
- WhileStmt
- ForStmt
- DoWhileStmt
- BinOp
- Literal
- Variavel

4.3 Análise semântica

A análise semântica é feita em `src/semantic_service.py`. Essa etapa verifica se o programa, além de estar sintaticamente correto, também faz sentido segundo as regras da linguagem.

Principais validações:

- variável não pode ser declarada duas vezes;
- variável precisa ser declarada antes de ser usada;
- atribuições precisam respeitar o tipo declarado;
- condições de `si`, `dum` e `fac-dum` precisam ser booleanas;

- operações matemáticas só podem ser feitas com tipos numéricos;
- o iterador do pro precisa ser do tipo `numerus`;
- os limites do pro precisam ser valores do tipo `numerus`.

4.4 Geração de código

A geração de código é feita em `src/generator_service.py`. O gerador percorre a AST e cria comandos equivalentes em Python.

Exemplos de tradução:

Latim	Python gerado
<code>scribe(x).</code>	<code>print(x)</code>
<code>lege(nome).</code>	<code>nome = input(...)</code>
<code>si (...) { ... }</code>	<code>if ...:</code>
<code>aliter { ... }</code>	<code>else:</code>
<code>dum (...) { ... }</code>	<code>while ...:</code>
<code>pro (i = 1 usque 5) { ... }</code>	<code>for i in range(1, 5 + 1):</code>
<code>fac { ... } dum (...).</code>	<code>while True: com break condicional</code>
.	

5. Especificação da Linguagem Latim

5.1 Tipos de dados

Tipo em Latim	Equivalente em Python	Descrição
<code>numerus</code>	<code>int</code>	Número inteiro
<code>decimalis</code>	<code>float</code>	Número decimal
<code>textus</code>	<code>str</code>	Texto/string
<code>veritas</code>	<code>bool</code>	Booleano
<code>inanis</code>	<code>None</code>	Valor nulo
<code>collectio</code>	<code>list</code>	Lista

5.2 Valores especiais

Latim	Python	Descrição
<code>verum</code>	<code>True</code>	Valor verdadeiro
<code>falsum</code>	<code>False</code>	Valor falso
<code>nihil</code>	<code>None</code>	Valor nulo

5.3 Operadores suportados

Operadores matemáticos

Operador	Descrição
<code>+</code>	Soma
<code>-</code>	Subtração
<code>*</code>	Multiplicação
<code>/</code>	Divisão

A gramática respeita a precedência matemática, ou seja, multiplicação e divisão são avaliadas antes de soma e subtração.

Exemplo:

resultado = 5 + 2 * 3.

Resultado esperado:

11

Operadores relacionais

Operador	Descrição
==	Igualdade
!=	Diferença
<	Menor que
>	Maior que
<=	Menor ou igual
>=	Maior ou igual

.

6. Sintaxe da Linguagem

6.1 Estrutura básica

principium

// comandos aqui

finis

6.2 Declaração de variáveis

numerus idade.

decimalis altura.

textus nome.

veritas ativo.

inanis vazio.

collectio notas.

Também é possível declarar mais de uma variável do mesmo tipo:

numerus x, y, soma.

6.3 Atribuição

idade = 20.

nome = "Gabriel".

ativo = verum.

vazio = nihil.

notas = [7, 8, 10].

6.4 Saída de dados

scribe("Olá mundo").

scribe(nome).

6.5 Entrada de dados

lege(nome).

lege(idade).

O gerador utiliza a tabela de símbolos para converter automaticamente a entrada de acordo com o tipo da variável. Por exemplo, variáveis numerus são convertidas com int() e variáveis decimalis com float().

6.6 Condicional

```
si (idade >= 18) {  
    scribe("Maior de idade").  
} aliter {  
    scribe("Menor de idade").  
}
```

6.7 Repetição com dum

```
dum (contador > 0) {  
    scribe(contador).  
    contador = contador - 1.  
}
```

6.8 Repetição com pro

```
pro (i = 1 usque 5) {  
    scribe(i).  
}
```

O intervalo é inclusivo. Portanto, pro (i = 1 usque 5) executa de 1 até 5.

6.9 Repetição com fac-dum

```
fac {  
    scribe("Executa pelo menos uma vez").  
} dum (falsum).
```

Essa estrutura equivale a um do while, pois o bloco é executado antes da condição ser testada.

.

7. Exemplo Completo

```
principium  
    numerus contador, limite.  
    decimalis media.  
    textus nome.  
    veritas executando.
```

```
    scribe("Digite seu nome: ").  
    lege(nome).
```

```
    limite = 3.  
    media = 5 + 2 * 3.  
    executando = verum.
```

```

si (media >= 10) {
    scribe("Media excelente!").
} aliter {
    scribe("Media dentro do esperado.").
}

pro (contador = 1 usque limite) {
    scribe(contador).
}

dum (limite > 0) {
    scribe(limite).
    limite = limite - 1.
}

fac {
    scribe("Executou pelo menos uma vez!").
    executando = falsum.
} dum (executando == verum).
finis

```

8. Como Instalar e Executar

8.1 Criar ambiente virtual

```
python -m venv venv
```

8.2 Ativar ambiente virtual

No Windows:

```
venv\Scripts\activate
```

No Linux/macOS:

```
source venv/bin/activate
```

8.3 Instalar dependências

```
pip install -r requirements.txt
```

9. Execução via Terminal

Para compilar um arquivo .latim:

```
python main.py programa.latim
```

O comando gera o arquivo:

```
saida.py
```

Depois, execute o Python gerado:

python saida.py

.

10. Execução pela Interface Gráfica

Também existe uma IDE desktop implementada em gui.py.

Para abrir:

python gui.py

A interface permite:

- escrever ou carregar código .latim;
- salvar arquivos;
- compilar e executar o código;
- visualizar mensagens de erro;
- visualizar o código Python gerado;
- usar entrada de dados por caixas de diálogo.

Em Linux ou WSL, pode ser necessário instalar o Tkinter:

`sudo apt-get update`

`sudo apt-get install -y python3-tk`

.

11. Testes

O projeto possui testes automatizados no arquivo tests/test_parser.py.

Para executar:

pytest tests/test_parser.py

Os testes verificam:

- criação correta da AST;
- reconhecimento de declarações;
- precedência de operadores;
- reconhecimento de condicionais;
- reconhecimento de laços pro e dum.

Também existem arquivos .latim usados como exemplos manuais:

Arquivo	Finalidade
programa.latim	Demonstração geral da linguagem
teste_decimal.latim	Teste com números decimais
teste_erro.latim	Teste de erros semânticos
teste_if.latim	Teste de estrutura condicional
teste_io.latim	Teste de entrada e saída
teste_loops.latim	Teste das estruturas de repetição

Arquivo	Finalidade
teste_math.latim	Teste de precedência matemática
teste_soma.latim	Teste simples de soma com entrada
erro_semantico.latim	Exemplo de incompatibilidade de tipos

12. Tratamento de Erros

O projeto diferencia erros sintáticos e semânticos.

12.1 Erro sintático

Acontece quando o código não segue a gramática da linguagem.

Exemplo:

```
numerus x
```

Nesse caso, falta o ponto final obrigatório após a declaração.

12.2 Erro semântico

Acontece quando o código está escrito em uma sintaxe válida, mas viola regras da linguagem.

Exemplo:

```
principium
numerus a.
a = "texto".
finis
```

A variável a foi declarada como numerus, mas recebeu um valor do tipo textus.

13. Desenvolvedores

- Gabriel Almeida - 12723112569
- Rafael Silva Rangel de Almeida – 1272412932
- Vitor Pio Vleira - 12725238440

14. Conclusão

O projeto demonstra, de forma prática, o funcionamento de um transpilador. A linguagem Latim possui sintaxe própria, tipos de dados, comandos de entrada e saída, condicionais, laços de repetição e validação semântica. O sistema transforma o código de entrada em uma AST e, a partir dela, gera código Python executável.

Além da execução via terminal, o projeto também possui uma interface gráfica que facilita a escrita, compilação, execução e visualização do código gerado.